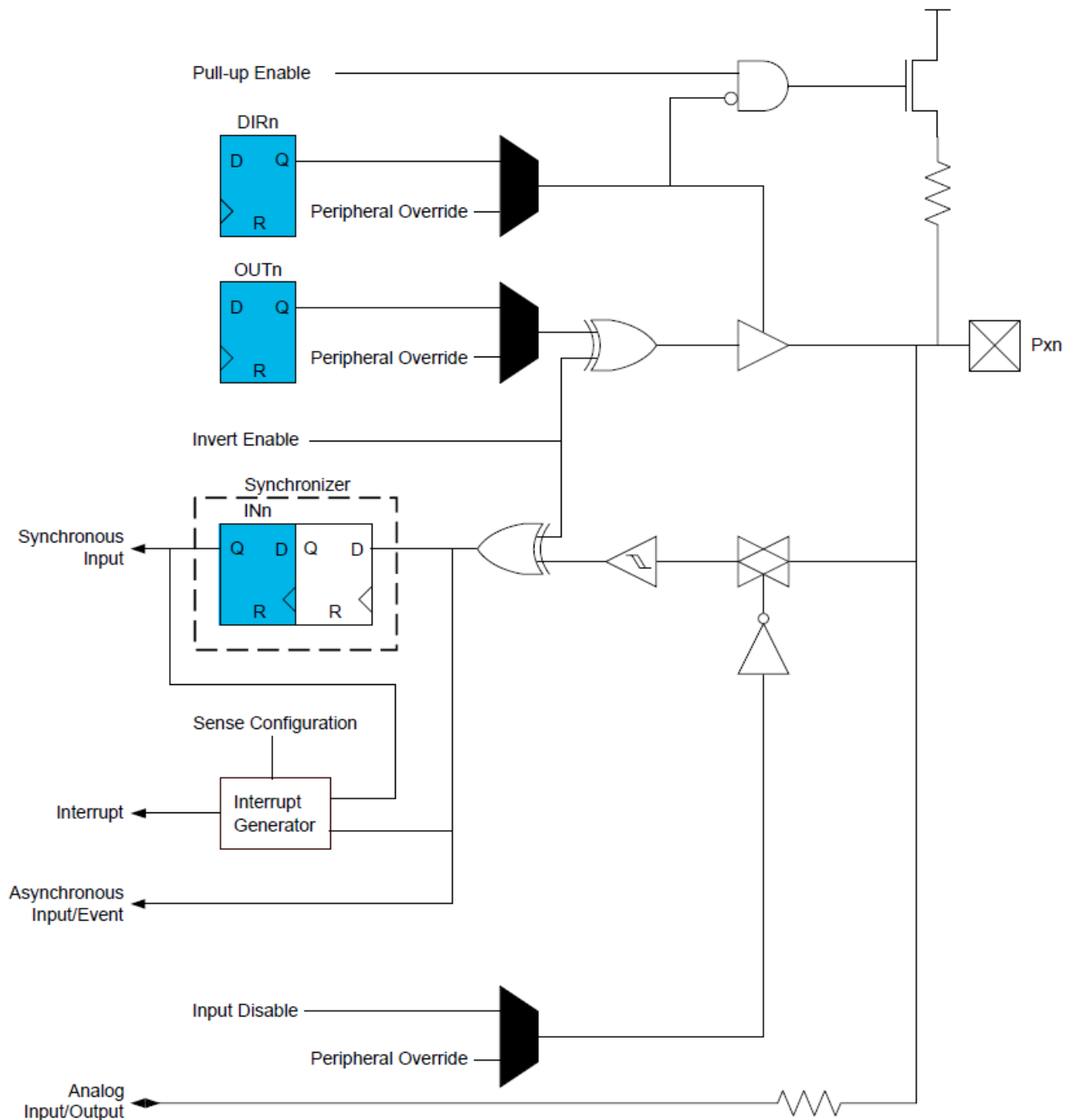


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 4 - GPIO at Register Level



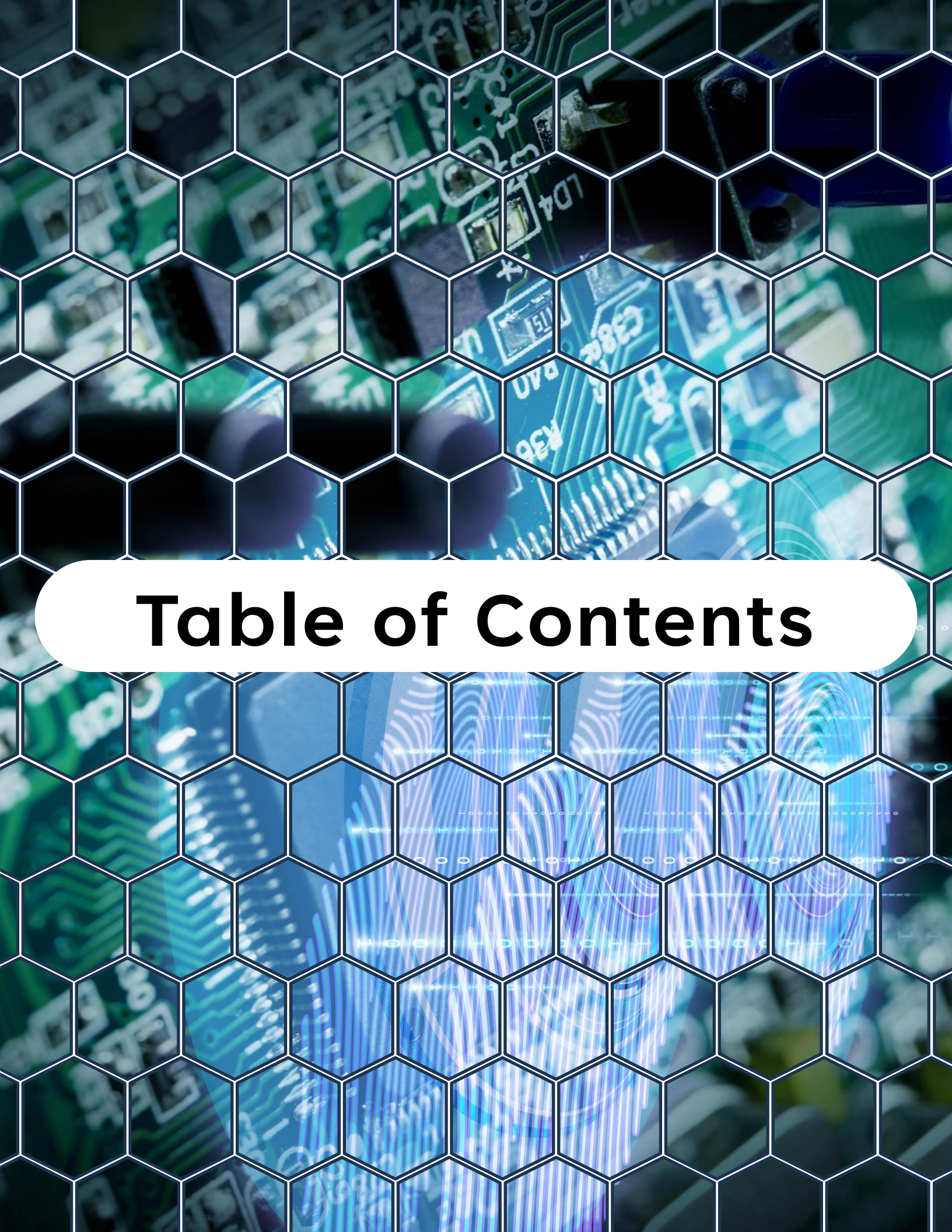
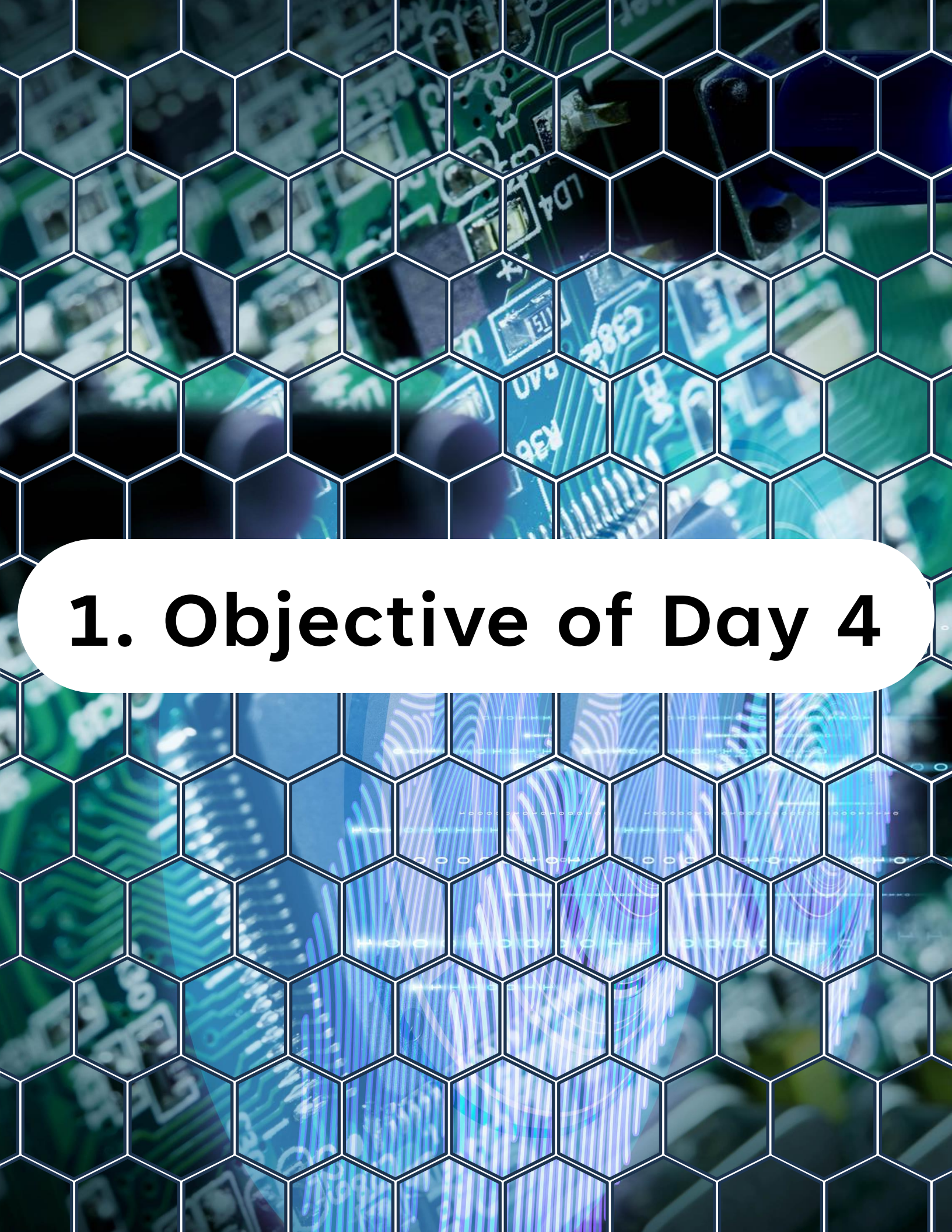


Table of Contents

Table of Contents

1. Objective of Day 4
2. GPIO Architecture Overview
3. The PORT Peripheral Model
4. Configuring Pin Direction
5. Driving Output Pins
6. Reading Input Pins
7. Pin Control Registers (PINnCTRL)
8. Atomic Pin Operations
9. GPIO Timing Characteristics
10. Practical Lab - LED Control Without Delays
11. Using GPIO as a Debug Tool
12. Common GPIO Mistakes
13. What You Learned Today
14. What Comes Next



1. Objective of Day 4

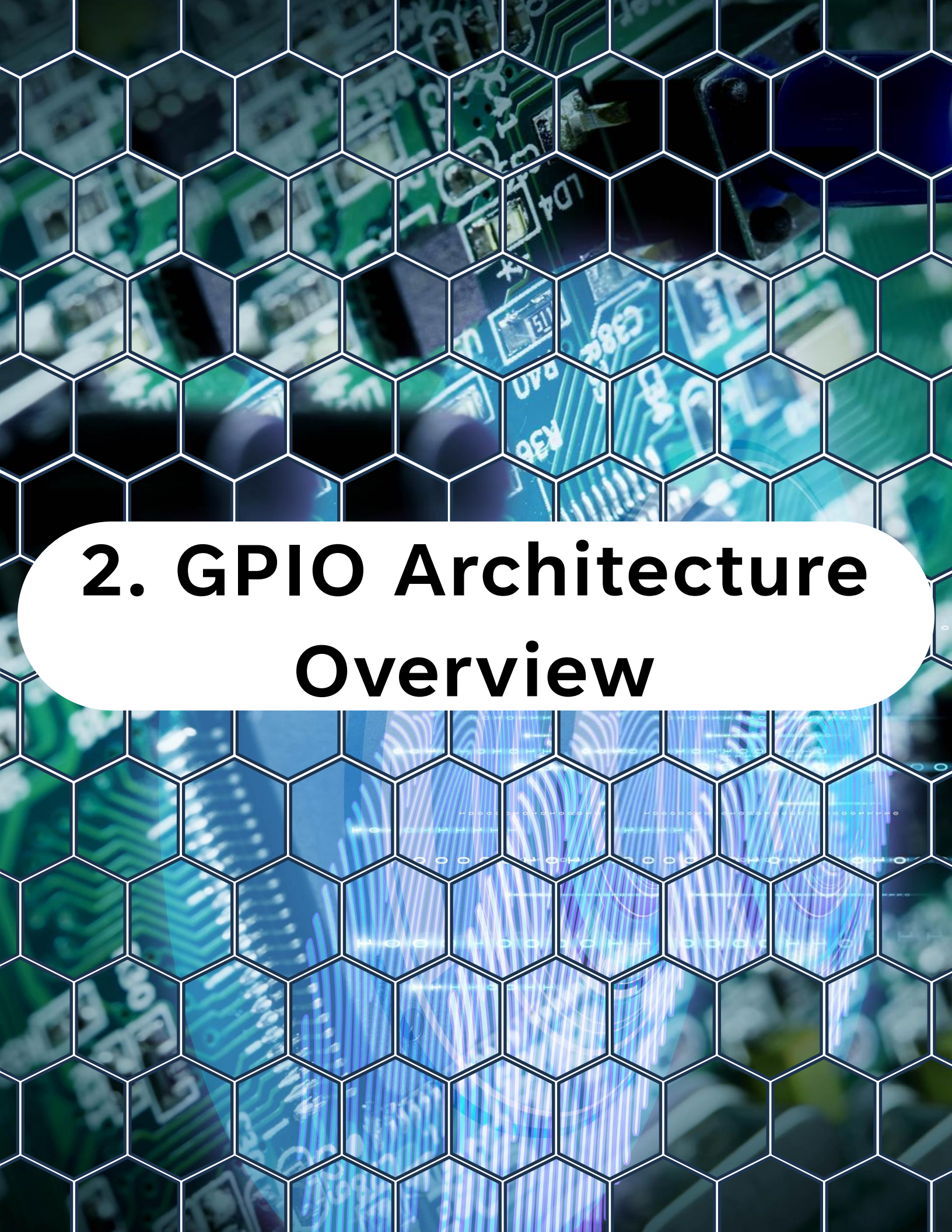
1. Objective of Day 4

GPIO is the first place where software physically touches the outside world.

Today you will learn how to control pins directly through registers, without helpers, delays, or abstractions. GPIO will also become your primary debugging tool for timing, state tracing, and sanity checks.

By the end of this day, you will be able to:

- Configure pins as input or output
- Drive pins high and low deterministically
- Use pull-ups and inversion correctly
- Read pin state reliably
- Use GPIO as a measurement and debug instrument



2. GPIO Architecture Overview

2. GPIO Architecture Overview

On the **AVR128DA48**, all digital I/O is handled by PORT peripherals.

Each port (PORTA, PORTB, PORTC, etc.) controls up to 8 pins. Every pin is fully configurable through memory-mapped registers.

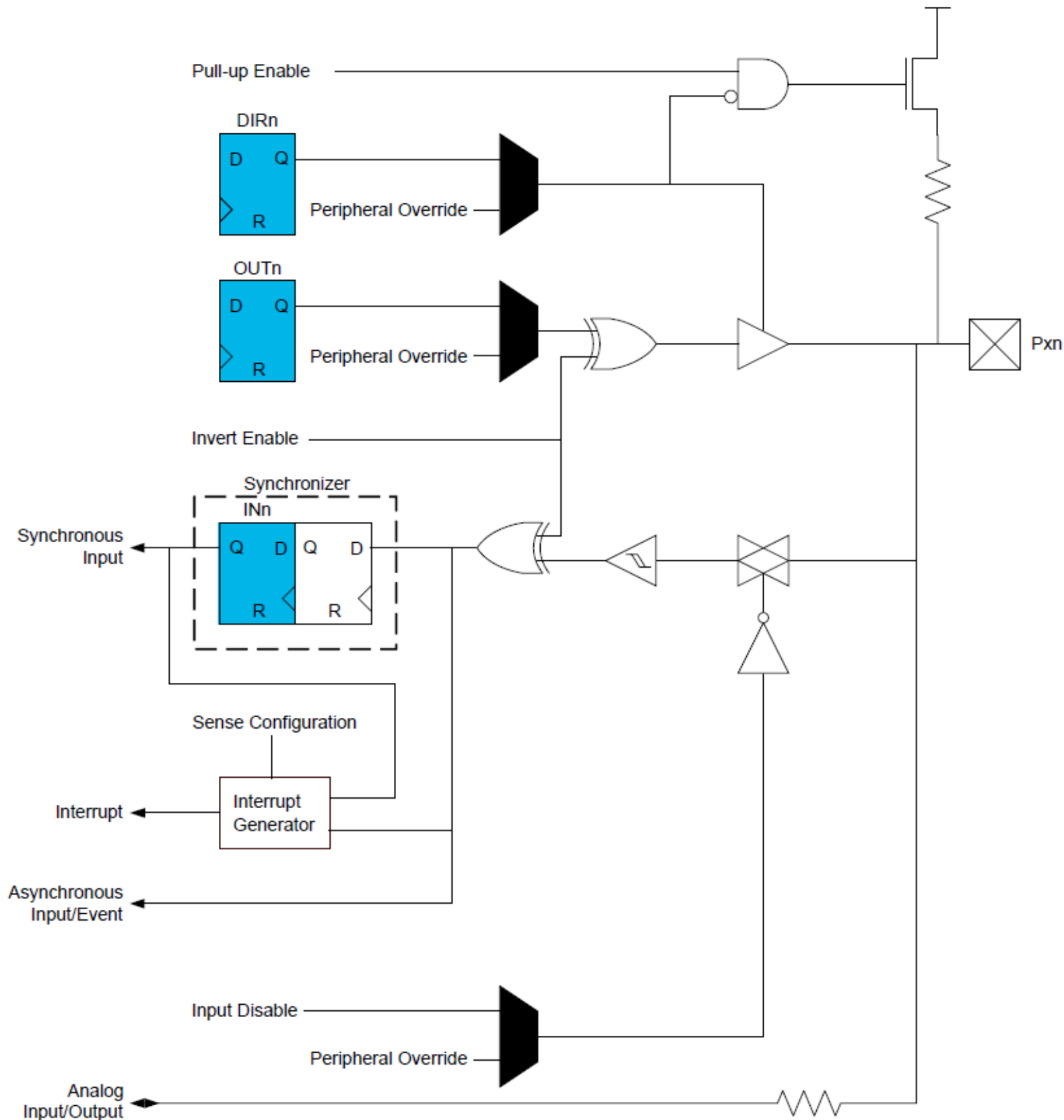
At a high level, each pin has:

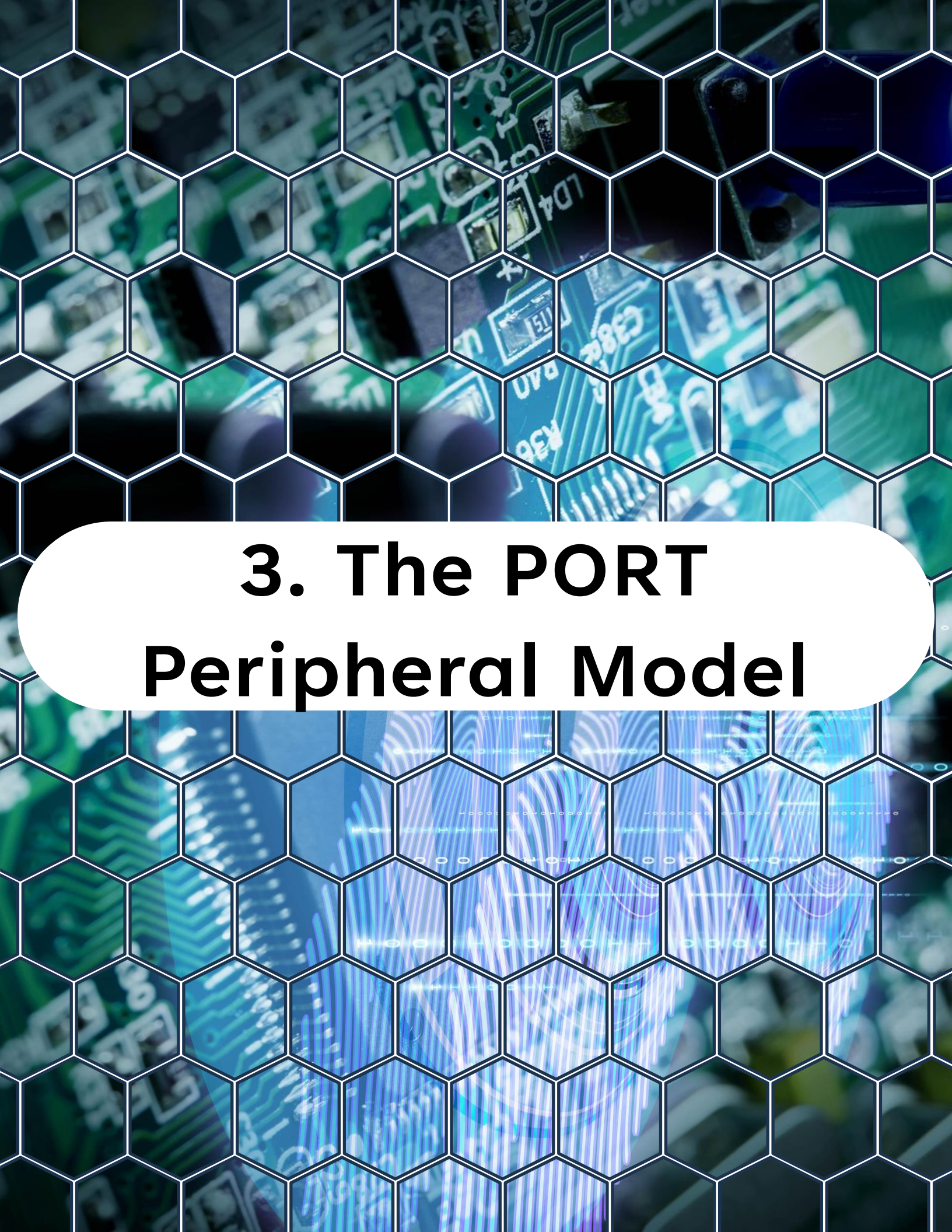
- Direction control
- Output latch
- Input sense
- Pin-specific configuration

There is no magic layer. Writing to a register drives real silicon.

2. GPIO Architecture Overview

PORT Block Diagram





3. The PORT Peripheral Model

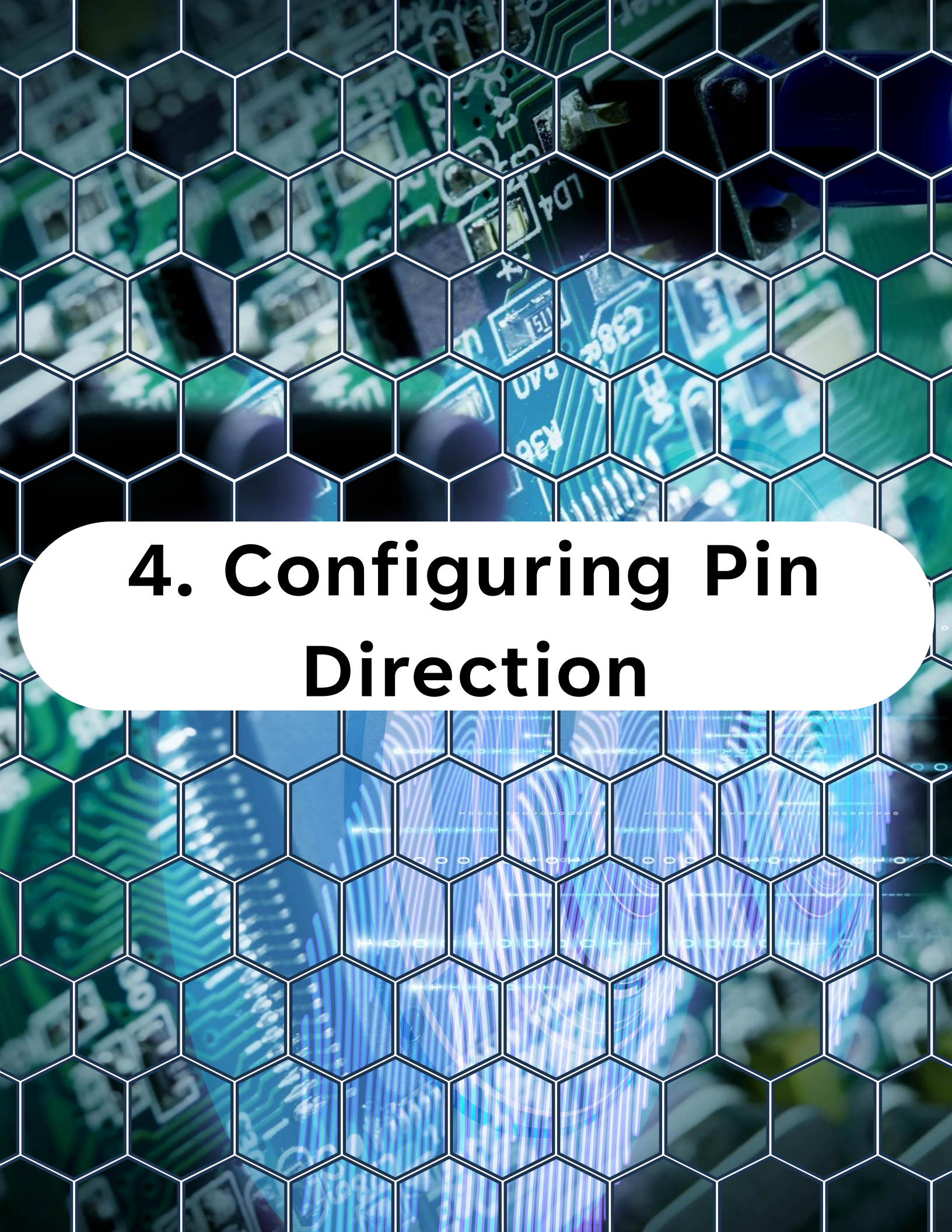
3. The PORT Peripheral Model

The I/O pins of the device are controlled by instances of the PORT peripheral registers. Each PORT instance has up to eight I/O pins. The PORTs are named **PORTA**, **PORTB**, **PORTC**, etc.

Each PORT instance exposes a consistent register set:

- **DIR**: Direction register (input or output)
- **OUT**: Output value register
- **IN**: Input value register
- **PINnCTRL**: Per-pin configuration registers
- **INTFLAGS**: Pin Interrupt Flag register

This consistency is intentional and makes the DA-series easy to learn once and reuse forever.



4. Configuring Pin Direction

4. Configuring Pin Direction

Output Pin

To configure a pin as output, set the corresponding bit in the register **DIR**.

Example: Setting Port A Bit 1 (PA1) to output

```
// Set the pin PA1 direction to output  
PORTA.DIR |= (1 << PIN1_bm); // PIN1_bm = 0x02
```

When this bit field is set to “1”, the Pxn is configured as an output pin, and the output driver is enabled.

Input Pin

For an input pin:

- Clear the corresponding DIR bit
- Optionally enable pull-up via PINnCTRL

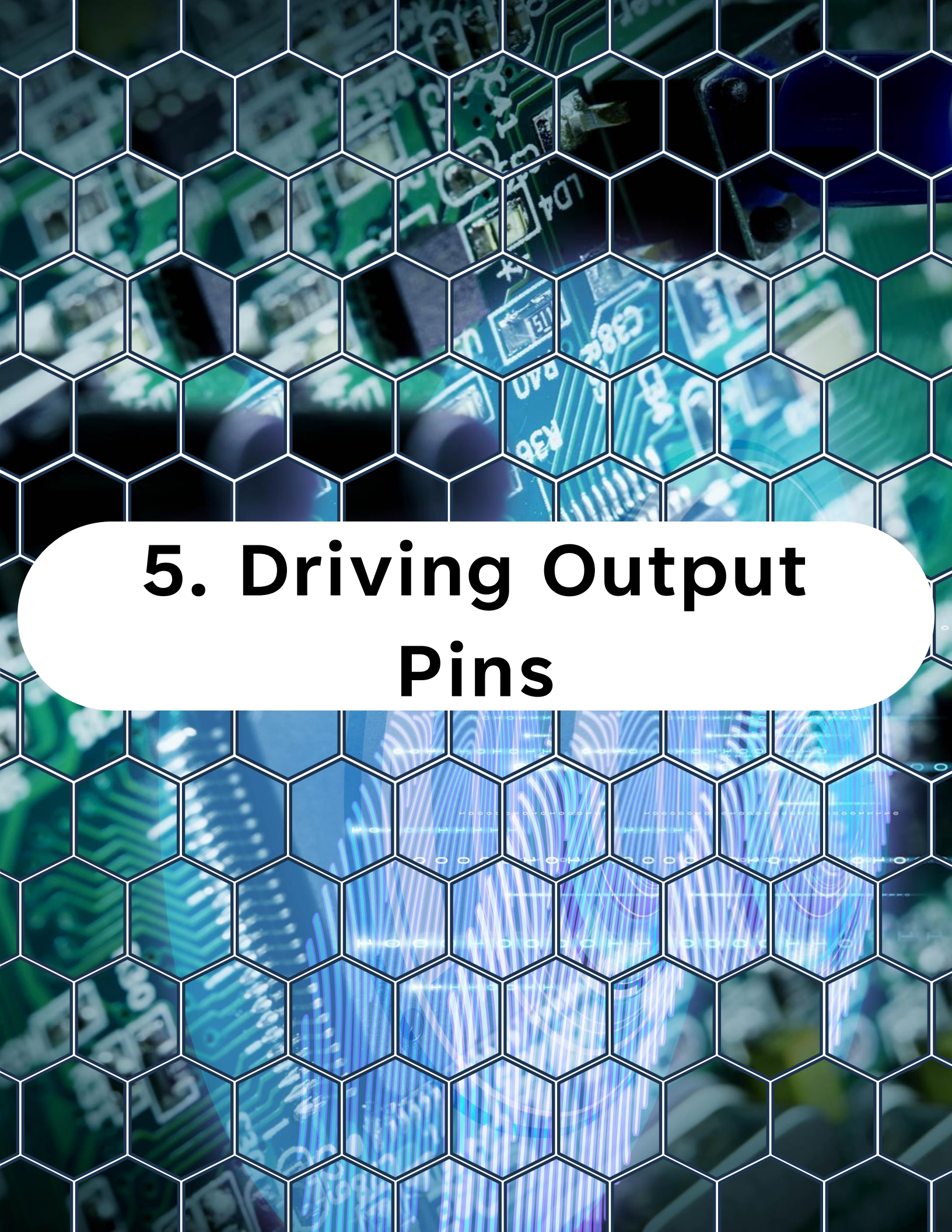
4. Configuring Pin Direction

Input pins are high-impedance unless a pull-up is enabled.

Example: Setting Port A Bit 1 (PA1) to input

```
// Set the pin PA1 direction to input
PORTA.DIR &= ~(1 << PIN1_bm);
// Enable pull-up
PORTA.PIN1CTRL |= (1 << PORT_PULLUPEN_bp);
```

When this bit field is set to “0”, the Pxn is configured as an input pin, and the output driver is disabled.



5. Driving Output Pins

5. Driving Output Pins

Once a pin is configured as an output:

- Writing 1 to **OUT** drives the pin high

```
// Set the pin PA1 direction to output
PORTA.DIR |= (1 << PIN1_bm);
// Drive the pin PA1 high
PORTA.OUT |= (1 << PIN1_bm);
```

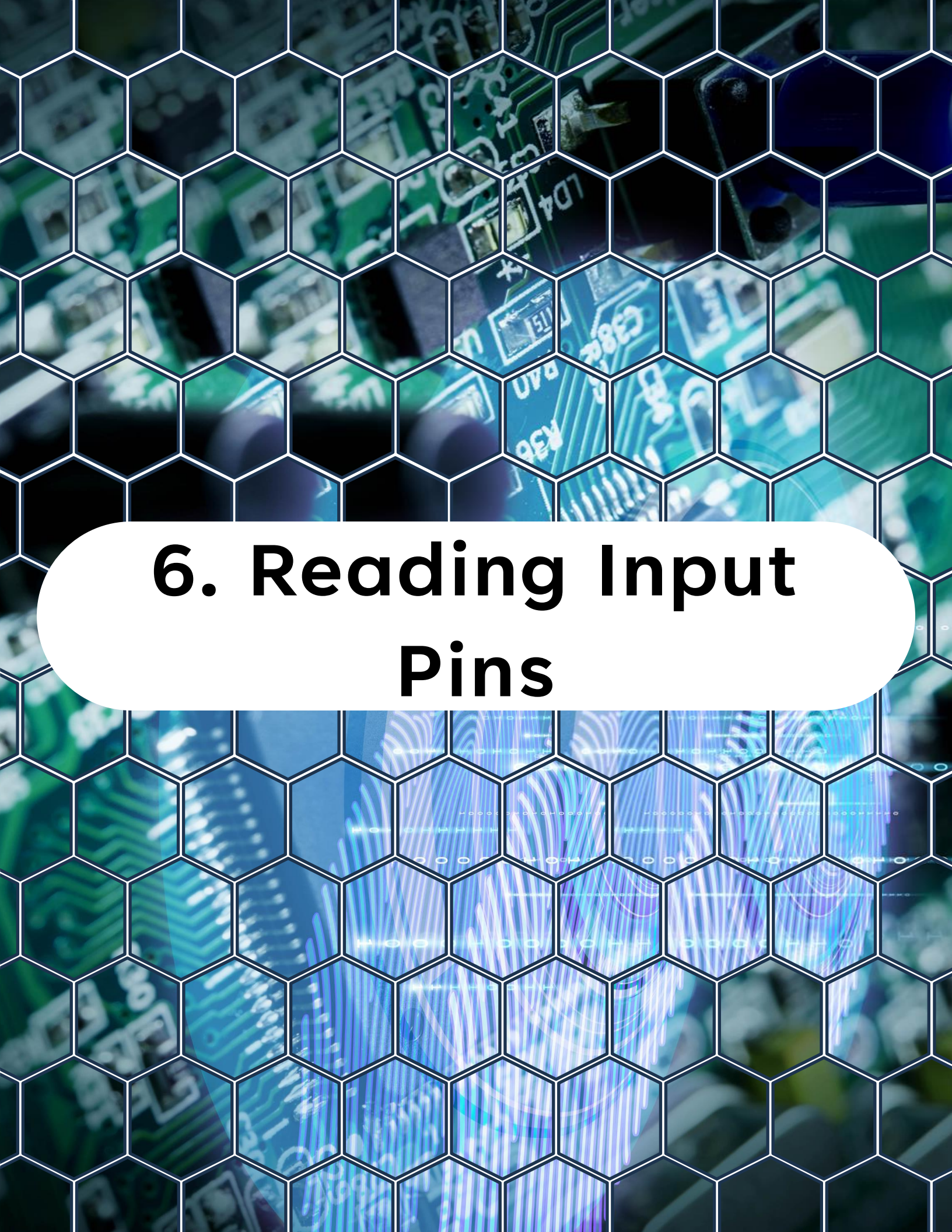
- Writing 0 to **OUT** drives the pin low

```
// Set the pin PA1 direction to output
PORTA.DIR |= (1 << PIN1_bm);
// Drive the pin PA1 low
PORTA.OUT &= ~(1 << PIN1_bm);
```

Important distinction:

- **OUT** is a latch, not the physical pin
- **IN** reflects the actual pin state

This matters when pins are shared or externally driven.



6. Reading Input Pins

6. Reading Input Pins

To read a pin:

- Access the **IN** register
- Mask the bit corresponding to the pin

```
static inline uint8_t pa1_read(void)
{
    return (PORTA.IN & PIN1_bm) ? 1u : 0u;
}
```

IN always reflects the voltage level (low or high) present on the pin, regardless of direction.

This allows:

- Reading back outputs for verification
- Detecting contention or external overrides



7. Pin Control Registers (PINnCTRL)

7. Pin Control Registers (PINnCTRL)

Each pin has its own control register: **PIN0CTRL**, **PIN1CTRL**, etc.

Common features include:

- Pull-up enable
- Input inversion
- Input sense configuration

```
// Configure PA1 as input
PORTA.DIRCLR = PIN1_bm;
// Enable internal pull-up resistor
PORTA.PIN1CTRL |= PORT_PULLUPEN_bm;
// Enable input inversion
PORTA.PIN1CTRL |= PORT_INVEN_bm;
// Sense both rising and falling edges
PORTA.PIN1CTRL |= PORT_ISC_BOTHEDGES_gc;
```

These registers are where GPIO becomes powerful and flexible.

7. Pin Control Registers (PINnCTRL)

Pull-Up Resistors

Internal pull-ups are enabled per pin.

Use cases:

- Buttons
- Open-drain signals
- Preventing floating inputs

Rule of thumb:

- Never leave an input floating
- Either drive it or pull it

7. Pin Control Registers (PINnCTRL)

Input Inversion

Input inversion flips the logical sense of the pin.

Example:

- Physical low -> logical 1
- Physical high -> logical 0

This is useful when:

- Hardware polarity is fixed
- You want cleaner software logic

Hardware inversion is often better than adding software if-statements everywhere.



8. Atomic Pin Operations

8. Atomic Pin Operations

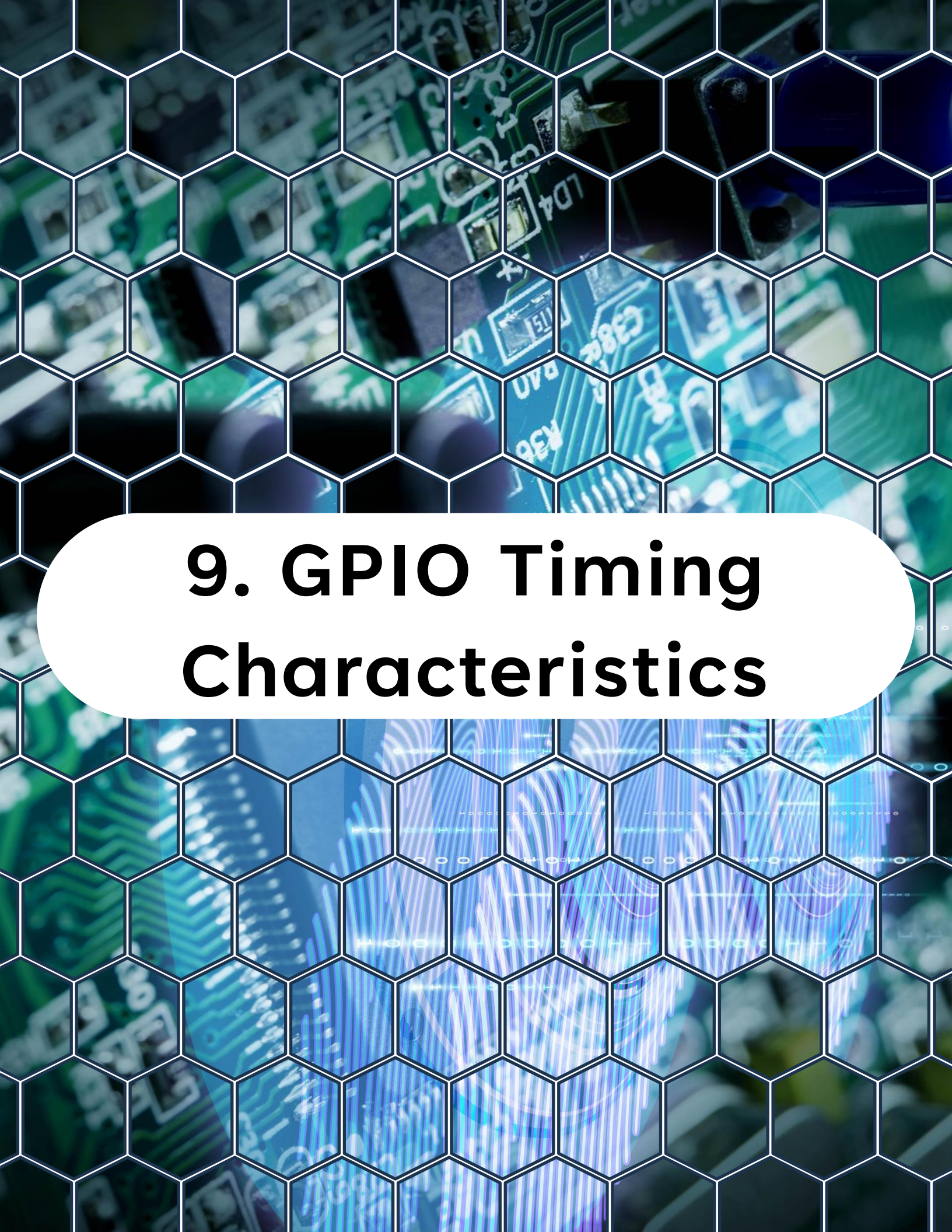
The DA-series provides atomic pin operations through special registers.

Why this matters:

- Avoid read-modify-write issues
- Safe operation inside ISRs
- Clean and deterministic behavior

You will rely on this later when GPIO is modified from multiple contexts.

```
// TCB0 ISR
ISR(TCB0_INT_vect)
{
    // Atomic: toggles only PC4, other pins unaffected
    PORTC.OUTTGL = PIN4_bm;
    // Clear interrupt flag
    TCB0.INTFLAGS = TCB_CAPT_bm;
}
```

9. GPIO Timing Characteristics

9. GPIO Timing Characteristics

GPIO is fast, but not infinite.

Important considerations:

- Output toggle speed depends on CPU clock
- Instruction count matters
- Debug builds are slower than release builds

This is why GPIO is such a good timing probe.



10. Practical Lab - LED Control Without Delays

10. Practical Lab - LED Control Without Delays

Lab Goal

Control an LED using pure GPIO and CPU execution speed.

No timers. No delay loops. Just raw instruction flow.

Step 1: Identify the LED Pin

Using the board schematic or pinout:

- Identify which PORT and pin drives the onboard LED
- Note if it is active-high or active-low

This matters. Do not guess.

10. Practical Lab - LED Control Without Delays

Step 2: Configure the Pin as Output

In main():

- Set the DIR bit
- Set a known initial state

This establishes deterministic startup behavior.

Step 3: Toggle the Pin in the Main Loop

Inside the while loop:

- Toggle the output pin continuously

Observe:

LED brightness (will appear dim or solid)

Oscilloscope or logic analyzer waveform if available

10. Practical Lab - LED Control Without Delays

This confirms:

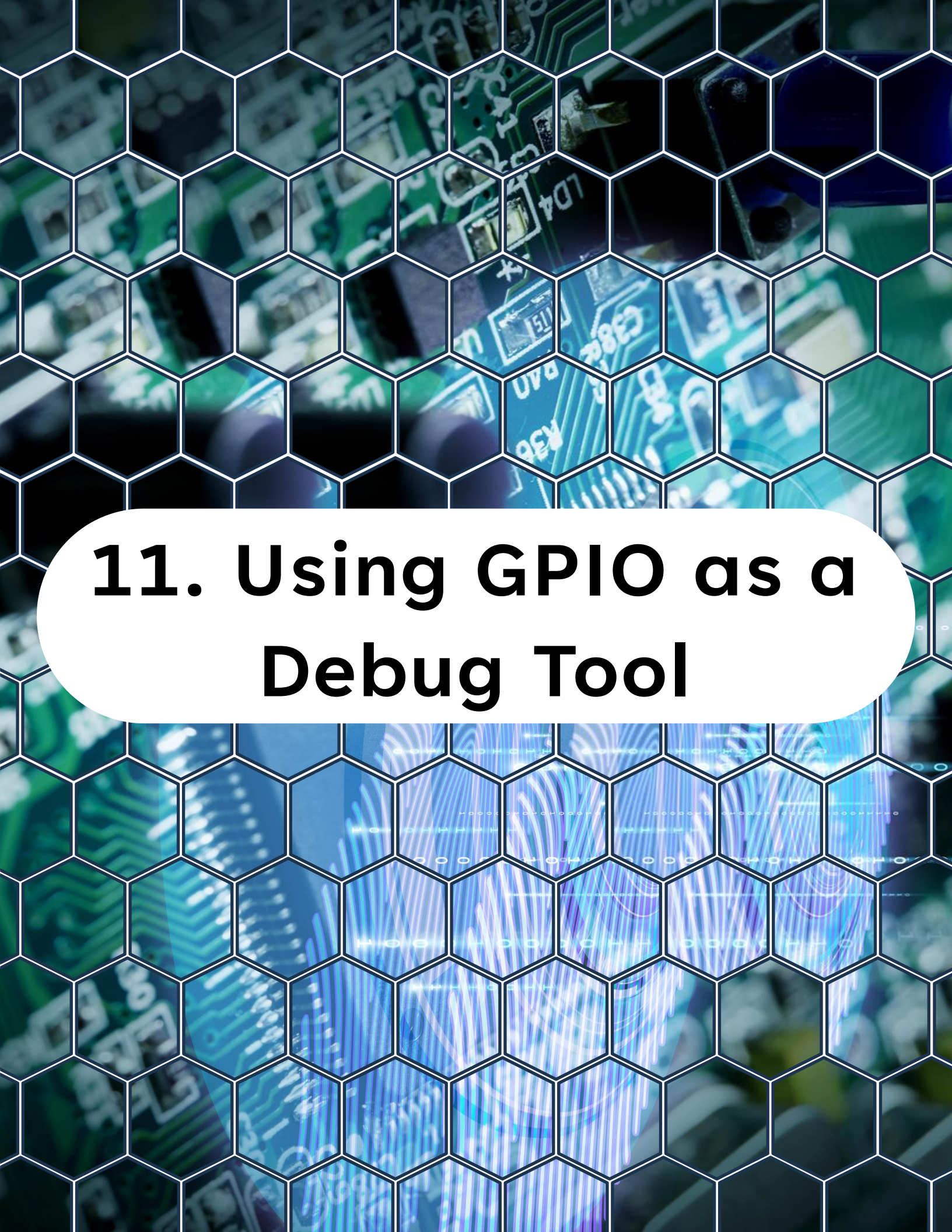
- GPIO is configured correctly
- Code is executing
- Clock configuration from Day 3 is working

Code Example

```
1  #include <avr/io.h>
2  #include <stdint.h>
3
4  /*
5   * This value is empirical.
6   * Change it and observe how blink speed changes.
7   * It directly depends on CPU clock and compiler optimization.
8   */
9  #define BLINK_TICKS  200000UL
10
11 /* ----- LED definition (from board schematic) ----- */
12
13 /* AVR128DA48 Curiosity Nano:
14  * LED is on PC6
15  * LED is active-low
16  */
17 #define LED_PORT      PORTC
18 #define LED_PIN_bm    PIN6_bm
19
20 /* ----- LED control helpers ----- */
```


10. Practical Lab - LED Control Without Delays

```
19
20 /* ----- LED control helpers ----- */
21
22 static inline void led_init(void) {
23     LED_PORT.DIRSET = LED_PIN_bm;    // Configure PC6 as output
24     LED_PORT.OUTSET = LED_PIN_bm;    // LED OFF (active-low)
25 }
26
27 static inline void led_toggle(void) {
28     LED_PORT.OUTTGL = LED_PIN_bm;    // Atomic toggle
29 }
30
31 /* ----- Main application ----- */
32
33 int main(void) {
34
35     uint32_t tick_counter = 0;
36
37     // Initialize the LED pin PC6
38     led_init();
39
40     while (1) {
41
42         tick_counter++;
43
44         if (tick_counter >= BLINK_TICKS) {
45             tick_counter = 0;
46             led_toggle(); // Toggle LED after N iterations
47         }
48     }
49 }
```



11. Using GPIO as a Debug Tool

11. Using GPIO as a Debug Tool

Professional embedded engineers use GPIO constantly for debugging:

- Toggle pin at ISR entry and exit
- Indicate state machine transitions
- Measure execution time
- Detect unexpected code paths

A single GPIO pin and a scope can outperform printf debugging in many cases.

Make this a habit early.



12. Common GPIO Mistakes

12. Common GPIO Mistakes

Watch out for:

- Forgetting to set DIR before writing OUT
- Leaving inputs floating
- Assuming LED polarity
- Using software delays to hide timing problems
- Mixing GPIO access styles inconsistently

GPIO bugs are simple, but they propagate everywhere if ignored.



13. What You Learned Today

13. What You Learned Today

By the end of Day 4, you should be comfortable with:

- PORT register structure
- Input vs output configuration
- Pull-ups and inversion
- Reading and writing pins correctly
- Using GPIO for debugging and timing

GPIO is simple, but mastery here makes every peripheral easier.



14. What Comes Next

14. What Comes Next

Day 5 - Reset System and Fuse Basics

Next, you will learn:

- How and why the MCU resets
- Different reset sources
- How to detect reset causes in firmware
- How fuses affect system behavior

Understanding reset behavior is critical for reliable embedded systems.

"Thanks for watching!

If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>